AI Forward Deployed Engineer

The AI Architect & FDE Course



Week 1 Day 3

3 weeks to Forward Deployed Engineer

Theory Practical Projects

Week 3: Become an AI FDE

- AI Solutions Architect
- Security, Guardrails, API Gateways
- Enterprise Context Layer & Integrations
- Production Deployment
- Capstone

Week 2: Become an AI Architect

- Architecture, Evals, Observability
- Agentic RAG, MCP, Memory
- Durable State & Human in the Loop
- Agent Teams, Swarms, Fleet
- Self-Improvement, RSI

Week 1: Become an AI Engineer

- Path to FDE
- Frontier LLMs & Agents
- Coding Agents & ADLC
- Open-Source, Local, Token Cost
- Harness Engineering

Taking stock

You can already:

- [x] Describe the role of the AI Forward Deployed Engineer
- [x] Explain the techniques that turn LLMs into AI products
- [x] Define an AI agent and an agent harness
- [x] Build an agent using LangChain

By the end of today, you'll be able to:

- [ ] Use coding agents like Claude Code or OpenCode
- [ ] Productively build software whilst staying accountable for your delivery
- [ ] Explain the ADLC, the Agentic Development Lifecycle
- [ ] Have hands-on experience delivering a pilot

If you're relatively new to coding agents, today is a crash course.
If not, speed me up to 2x!

Recap

Frontier LLMs and agents on Day 2



How AI products work
Six tricks turn next-token prediction into a product like ChatGPT



The frontier
Picking a model means trading off intelligence, speed and cost



Agents
An LLM with tools in a loop to achieve a goal



The first lab
LLMs and a LangChain agent decided 25 UnderCover claims

Today Claude Code builds a pilot of UnderCover

Setting the scene

Andrej Karpathy coined “vibe coding” in February 2025



2025 was an emotional rollercoaster

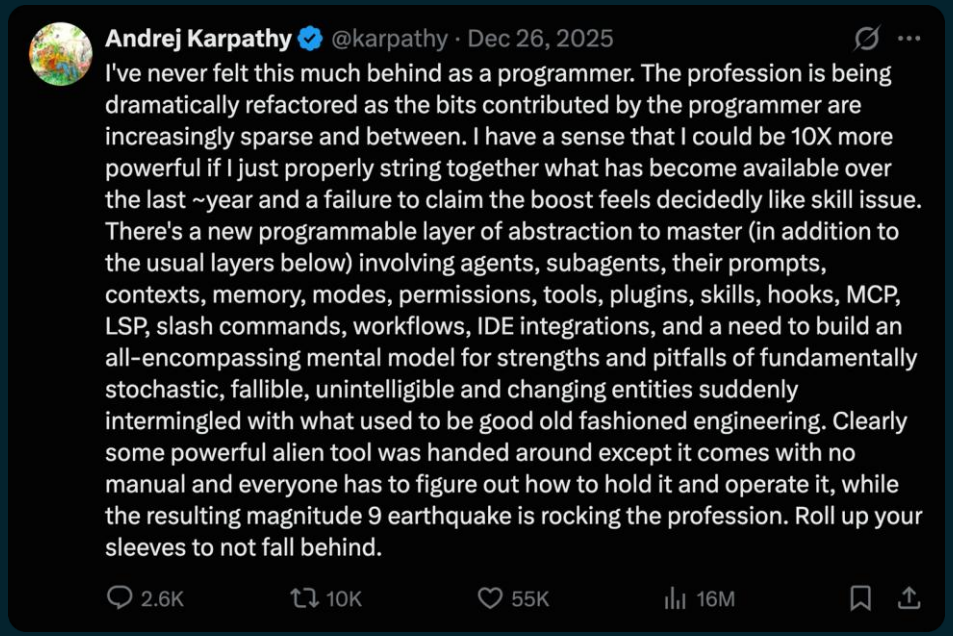
Surprise, denial, astonishment, frustration, anger, acceptance

There was an inflection point in November 2025

When Claude Opus 4.5 arrived

By the end of 2025

"Some powerful alien tool was handed around except it comes with no manual"



Andrej Karpathy @karpathy · Dec 26, 2025

I've never felt this much behind as a programmer. The profession is being dramatically refactored as the bits contributed by the programmer are increasingly sparse and between. I have a sense that I could be 10X more powerful if I just properly string together what has become available over the last ~year and a failure to claim the boost feels decidedly like skill issue. There's a new programmable layer of abstraction to master (in addition to the usual layers below) involving agents, subagents, their prompts, contexts, memory, modes, permissions, tools, plugins, skills, hooks, MCP, LSP, slash commands, workflows, IDE integrations, and a need to build an all-encompassing mental model for strengths and pitfalls of fundamentally stochastic, fallible, unintelligible and changing entities suddenly intermingled with what used to be good old fashioned engineering. Clearly some powerful alien tool was handed around except it comes with no manual and everyone has to figure out how to hold it and operate it, while the resulting magnitude 9 earthquake is rocking the profession. Roll up your sleeves to not fall behind.

2.6K 10K 55K 16M

The coding agent landscape

Three ways to write software with a coding agent



IDE

An editor built for agents

Cursor
Antigravity
Kiro



Plugin

An agent added to your editor

GitHub Copilot



CLI

An agent in your terminal

Claude Code
Codex CLI
Gemini CLI
Cursor CLI

OpenCode
Pi

The lines blur: Claude Code also runs inside VS Code and as a desktop app

The rise of Claude Code

Late 2024 — Created by Anthropic Engineer Boris Cherny, as a side-project

Apr 2025 — General release, the first of the CLI based AI Coding Assistants

Sep 2025 — V2 brings VS Code extension, robust ecosystem (checkpoints, subagents, hooks, skills, background tasks, SDK …)

Nov 2025 — Opus 4.5 released, considered inflection point; $1B run-rate revenue ($2.5B by Feb)

Jan 2026 — Cowork released (now renamed)

Coding Agents in this course

Pick one for this course



Claude Code
Anthropic



Codex
OpenAI



OpenCode
Open source

Other good options



GitHub Copilot



Cursor



Pi

These are also called "Coding Agent Harnesses";
Claude Code is the Coding Agent Harness and Claude Opus 5.5 is the LLM

OpenCode and OpenCode Zen

The free route through this course



OpenCode

What it is
An open-source coding agent for the terminal, the desktop and your IDE

Models
Works with almost any provider, including OpenRouter and local models

How it works
Very consistent; reads AGENTS.md by default



OpenCode Zen

What it is
A set of models the OpenCode team has tested for coding agents

Pricing
Pay as you go, with a list of free models that changes often

Free models
Your data may be used to improve the model

In OpenCode, /connect adds Zen and /models lists today's free models
It's a wild west and can be extremely frustrating! Tips for success coming later..

Installing your Coding Agent Harness

Pick one of the main 3

Claude Code

0 **Sign up**

`claude.ai`

1 **Install**

`claude.com/claude-code`

2 **Start it**

```
claude
```

3 **Sign in**

```
/login
```

Codex

0 **Sign up**

`chatgpt.com`

1 **Install**

```
npm i -g @openai/codex
```

2 **Sign in with ChatGPT**

```
codex login
```

3 **Start it**

```
codex
```

OpenCode

0 **Sign up for Zen**

`opencode.ai/auth`

1 **Install**

```
npm i -g opencode-ai
```

2 **Start it**

```
opencode
```

3 **Pick OpenCode Zen**

```
/connect
```

Be the boss

Direct the agent and stay accountable for the result



Give it the brief

The spec, the style and the success criteria, concisely, in markdowns



Start simple and build in phases

A plan with measurable success criteria for each phase



Challenge it and ask for evidence

Do not get lazy: make it show the test run, the diff and the screenshot



You own what ships

The PR carries your name, whoever wrote the code

You own the code you deliver. Never push slop.

AGENTS.md

Markdown instructions the agent reads at the start of every session

```
# UnderCover
## Goals
..
## Success criteria
..
# Code standards
See docs/CODE_STANDARDS.md
```

Pro-tips

Be clear
Be concise
Favor positives over negatives
Occasionally use IMPORTANT

but most importantly:
EXPERIMENT!

Claude Code, Codex and OpenCode all read AGENTS.md

Best practice: keep AGENTS.md short and reference other files from it

Permission modes

Shift+Tab switches mode during a Claude Code session



Manual
Asks before it edits files, runs commands or reaches the network



Accept edits
Edits files and runs simple file commands without asking



Plan
Reads and explores, then proposes a plan for you to approve



Auto the default in recent versions
A second model checks each action in place of you



Bypass permissions
Everything runs without asking: only for containers and VMs

And add allow and deny rules in .claude/settings.json

Tips for success

Best practices for working with coding agents



Clear the context
A fresh session per task and /compact before it degrades



Plan first
Ask for a plan and agree it before it writes any code



Verifiable phases
Success criteria it checks before the next phase



Checkpoints
Commit often, and use /rewind or git to revert



Ask for options
Two or three approaches before it builds one



Keep notes
A progress file in the repo for the next session

The Model Context Protocol

An open standard for connecting AI applications to external systems



A standard way to use someone else's tool
“The USB-C port for AI applications”



A standard way to share your tool with someone else
Like making an API, but without needing an API



Now ubiquitous across Coding Agents and applications
Including Claude Code, Codex and OpenCode

Connecting an MCP server

Set up either with slash commands or at the command line

1

Add the Playwright server for today's lab

```
claude mcp add playwright -- npx -y @playwright/mcp@latest
```

2

Choose the scope

Add -s project and it is specific to this project in the repo for all the team.
The default is local: you, on this machine.

3

Check it and sign in

```
claude mcp list
```

/mcp inside the session shows the same list and handles OAuth for a remote server

```
Codex      codex mcp add playwright -- npx -y @playwright/mcp@latest
OpenCode   opencode mcp add, or an mcp entry in opencode.json
```

The 2 meanings of ADLC

Quite confusing!

MOST OF THIS COURSE



1. Building agents with code

The emerging process for building AI agents including risks & controls, evaluations, adoption, observability and maintenance.

LangChain: Agent Development Lifecycle · Microsoft Foundry: agent lifecycle

TODAY



2. Building code with agents

The emerging process for writing software with agents like Claude Code while staying accountable for the results.

AWS: AI-Driven Development Life Cycle · GitHub Spec Kit SDD

The Agentic Development Lifecycle

Coding Agents have changed the way we work, and it's still evolving

Platform



Harness

The Agent

State and memory

Skills and integrations

Sub-agents and agent teams

Mostly settled

Process



Harden

The Environment

Hooks, linters and CI/CD

Sandboxes and gateways

Tests and evaluations

Partly settled

People



Humans

You

Named accountability

Meaningful review

The end-to-end process

Mostly open

The UnderCover repo

The backend, agent and dashboard start as a hello world

```
undercover/
├── AGENTS.md              Claude Code reads this first
├── docs/
│   ├── SPEC.md            what the business needs
│   ├── TECH_STACK.md      the tech choices
│   ├── CODE_STANDARDS.md  how to write the code
│   ├── PROCESS.md         how to plan and build
│   ├── BRAND.md           the look
│   └── STATUS.md          what's built so far
├── backend/               FastAPI and SQLite
├── agent/                 LangChain
├── dashboard/             React, Vite and TypeScript
├── e2e/                   Playwright
├── data/                  UnderCover's records
├── labs/                  the lab notebooks
├── pyproject.toml         Python packages and settings
├── package.json           Node packages and scripts
└── lefthook.yml           the pre-commit hook
```

The brief

The product

Data and labs

Packages and checks

The controls built into the repo

Every change has to pass these checks



Formatting and linting
ruff for Python, Prettier and ESLint for TypeScript



Type checking
pyright for Python, tsc in strict mode for TypeScript



Unit tests
pytest and vitest, with at least 80% coverage



End-to-end tests
Playwright starts the backend and the dashboard and tests them in a browser



The pre-commit hook
lefthook runs all of these except Playwright on every commit

PROCESS.md gives Claude Code instructions so that this is never bypassed

What the linters check

ruff and ESLint run all of these on every commit

`ruff check`
Python

pycodestyle and **Pyflakes**: style, unused and undefined names
isort and **pyupgrade**: import order and newer syntax
bugbear and **simplify**: likely bugs and simpler code
bandit: security
Pylint: code smells, with at most 5 arguments and 12 branches
Limits: complexity 10 and 5 nested blocks
pytest style and **FastAPI**: test and API conventions

`eslint`
TypeScript

typescript-eslint strict and stylistic: type-aware rules
SonarJS: bugs and code smells
React Hooks and **React Refresh**: React rules
jsx-a11y: accessibility
Vitest and **Playwright**: test conventions
Limits: complexity 15, depth 4, 5 parameters, 200 lines a function

Working in the repo

Run these in a terminal in the undercover folder

```
uv sync
```

Install the Python packages

You ran this on Day 1

```
npm install
```

Install the Node packages

Includes the pre-commit hook and Playwright's browser

```
npm run dev
```

Start the backend and the dashboard

Then open `localhost:5173` in your browser

```
npm run check
```

Run every check

Python, TypeScript and then the end-to-end tests

```
git commit
```

Commit a change

The pre-commit hook formats and checks it first

The three pieces of UnderCover

Requirements in `docs/SPEC.md` approach in `docs/TECH_STACK.md`



Dashboard

React, Vite, TypeScript

The claims queue
What the agent is doing
Its instructions, read only

Backend

FastAPI and SQLite

An OpenAPI document
The seed data and the queue
Runs and decisions

Agent

LangChain

Day 2's agent, now in the repo
One entry point per claim
Its tools call the backend

The dashboard and the agent only talk to the backend's API

Today’s lab, step by step

Claude Code builds UnderCover from `docs/SPEC.md`

1. **Install Claude Code** `claude.com/claude-code`
2. **Log in to Anthropic** Run `claude` and then `/login`
3. **Switch to a new branch** `git checkout -b week1day3-yourname`
4. **Ask Claude for a plan** `Please plan the pilot`
5. **Check the plan** Read the phases and success criteria in `docs/PLAN.md`
6. **Ask Claude to build it** `Please implement the plan for the pilot`
7. **Review code and test results** Scripts, a human and an LLM
8. **Test the product** The dashboard, the API and the agent on a claim

Tips for success with free coding agents

Free models need more guidance than Claude Code



Very small incremental steps
Ask for one small change at a time



Test at every point
Run `npm run check` after every change



Commit at every point
Go back as needed and try again



Patience and tenacity
Free models often need a second or third try

Or simply: `git checkout week1day3-end` *to get my version*

Where do humans fit in?

It's still evolving, but this is the state of play



Accountability
The buck stops with you; the LLM is the tool



Evaluations
You decide how to measure success



Judgement
Trade-offs and nuanced decisions



Taste
Clumsy style, over-engineering, overly verbose, ...



Harness
You own the scaffolding around the LLM

How to share your work

Send me a write-up of your pilot as a pull request

1. **Make your folder** `community_contributions/your-name`
2. **Write PILOT.md in it** What you did, your takeaways and a link to your repo
3. **Share your plan too** Copy your `docs/PLAN.md` into the folder if you like
4. **Fork my repo on GitHub** Then run these in the undercover folder

```
git remote add fork <your fork URL>          Your fork on GitHub
git push fork main                           Publish your build
git checkout -b my-pilot origin/main         A branch from my main
git add community_contributions/your-name    Only your folder
git commit -m "Add my pilot"                 Commit your write-up
git push fork my-pilot                       Then open a pull request
```

Post about it on LinkedIn and tag me. I'll weigh in to amplify your achievement

3 weeks to Forward Deployed Engineer

Theory Practical Projects

Week 3: Become an AI FDE

AI Solutions Architect	Security, Guardrails, API Gateways	Enterprise Context Layer & Integrations	Production Deployment	Capstone

Week 2: Become an AI Architect

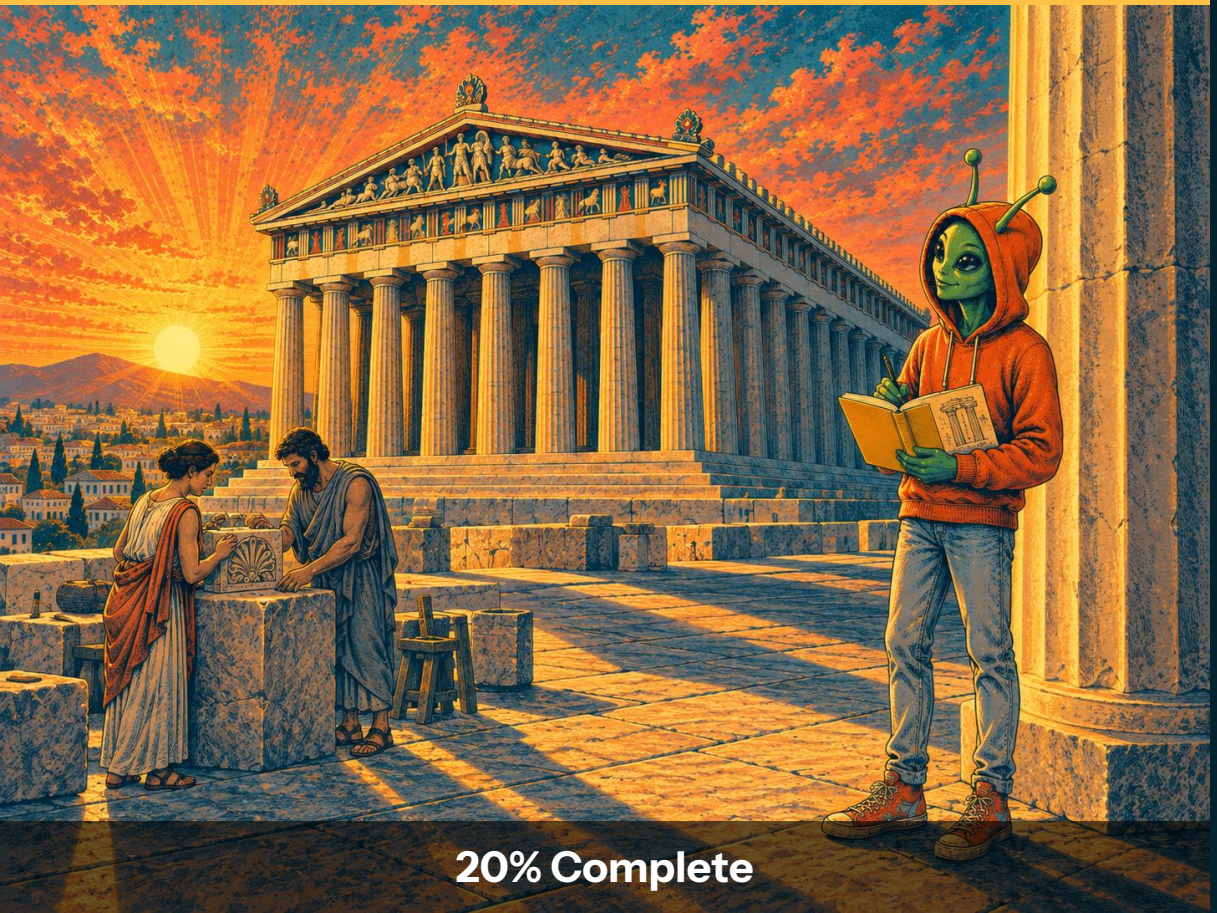
Architecture, Evals, Observability	Agentic RAG, MCP, Memory	Durable State & Human in the Loop	Agent Teams, Swarms, Fleet	Self-Improvement, RSI

Week 1: Become an AI Engineer

Path to FDE	Frontier LLMs & Agents	Coding Agents & ADLC	Open-Source, Local, Token Cost	Harness Engineering

AI Forward Deployed Engineer

The AI Architect & FDE Course



20% Complete